

Hands-on Activity No. <5.1>

<Multidimensional Arrays>

Course Code: CPE007

Program: Computer Engineering

Course Title: Programming logic and design

Date Performed: 9/29/2025

Section: CPE11S1

Date Submitted: 9/30/2025

Name(s): Hanz Rhayven M, Pabalan

Instructor: Engr. Jimlord Quejado

6. Output

*] multiplication table.cpp

```
1  #include <iostream>
2  #include <iomanip>
3  using namespace std;
4
5  int main () {
6      const int SIZE = 10;
7      int table [SIZE][SIZE];
8
9
10     for (int i = 0; i < SIZE; i++){
11         for (int j = 0; j < SIZE; j++) {
12             table[i][j] = (i + 1) * (j + 1);
13         }
14     }
15
16     cout<<"Multiplication Table:\n\n";
17     cout<<" ";
18     for (int j = 1; j <= SIZE; j++) {
19         cout<< setw(4) << j <<" |";
20     }
21     cout<<"\n-----\n";
22
23     for (int i = 0; i < SIZE; i++) {
24         cout<< setw(4) << i + 1 <<" |";
25         for (int j = 0; j < SIZE; j++) {
26             cout << setw(4) <<table[i][j] <<" |";
27         }
28         cout<< "\n ----- \n";
29     }
30
31     return 0;
32 }
```

OUTPUT:

```
C:\Users\hanz\OneDrive\Docu X + v
Multiplication Table:
 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
--|---|
1 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
--|---|
2 | 2 | 4 | 6 | 8 | 10 | 12 | 14 | 16 | 18 | 20 |
--|---|
3 | 3 | 6 | 9 | 12 | 15 | 18 | 21 | 24 | 27 | 30 |
--|---|
4 | 4 | 8 | 12 | 16 | 20 | 24 | 28 | 32 | 36 | 40 |
--|---|
5 | 5 | 10 | 15 | 20 | 25 | 30 | 35 | 40 | 45 | 50 |
--|---|
6 | 6 | 12 | 18 | 24 | 30 | 36 | 42 | 48 | 54 | 60 |
--|---|
7 | 7 | 14 | 21 | 28 | 35 | 42 | 49 | 56 | 63 | 70 |
--|---|
8 | 8 | 16 | 24 | 32 | 40 | 48 | 56 | 64 | 72 | 80 |
--|---|
9 | 9 | 18 | 27 | 36 | 45 | 54 | 63 | 72 | 81 | 90 |
--|---|
10 | 10 | 20 | 30 | 40 | 50 | 60 | 70 | 80 | 90 | 100 |
-----
Process exited after 1.974 seconds with return value 0
Press any key to continue . . . |
```

```
#include <iostream>
using namespace std;

const int SIZE = 3;

void displayBoard(char board[SIZE][SIZE]) {
    cout << "\n";
    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            cout << " " << board[i][j];
            if (j < SIZE - 1) cout << " | ";
        }
        cout << "\n";
        if (i < SIZE - 1) cout << "----+----+----\n";
    }
    cout << "\n";
}

bool isValidMove(char board[SIZE][SIZE], int row, int col) {
    return (row >= 0 && row < SIZE && col >= 0 && col < SIZE && board[row][col] == ' ');
}

bool checkWin(char board[SIZE][SIZE], char player) {
    // Check rows and columns
    for (int i = 0; i < SIZE; i++) {
        if (board[i][0] == player && board[i][1] == player && board[i][2] == player) return true;
        if (board[0][i] == player && board[1][i] == player && board[2][i] == player) return true;
    }
    // Check diagonals
    if (board[0][0] == player && board[1][1] == player && board[2][2] == player) return true;
    if (board[0][2] == player && board[1][1] == player && board[2][0] == player) return true;

    return false;
}

int main() {
    char board[SIZE][SIZE] = {
```

2.

```

multiplication table.cpp  tictactoe.cpp
34 }
35
36 int main() {
37     char board[SIZE][SIZE] = {
38         {' ', ' ', ' ', ' ', ' '},
39         {' ', ' ', ' ', ' ', ' '},
40         {' ', ' ', ' ', ' ', ' '},
41     };
42
43     char currentPlayer = 'X';
44     int moves = 0;
45     bool gameOver = false;
46
47     while (moves < SIZE * SIZE && !gameOver) {
48         displayBoard(board);
49
50         int row, col;
51         cout << "Player " << currentPlayer << ", enter row and column (0-2): ";
52         cin >> row >> col;
53
54         if (isValidMove(board, row, col)) {
55             board[row][col] = currentPlayer;
56             moves++;
57
58             if (checkWin(board, currentPlayer)) {
59                 displayBoard(board);
60                 cout << "Player " << currentPlayer << " WINS! \n";
61                 gameOver = true;
62             } else if (moves == SIZE * SIZE) {
63                 displayBoard(board);
64                 cout << "It's a DRAW! \n";
65             } else {
66                 currentPlayer = (currentPlayer == 'X') ? 'O' : 'X'; // Switch player
67             }
68         } else {
69             cout << "Invalid move! Try again.\n";
70         }
71     }
72
73     return 0;
74 }

```

OUTPUT:

```
C:\Users\hanz\OneDrive\Doc x + v
X | X |
---+---+---
  | O |
---+---+---
O |   | X

Player 0, enter row and column (0-2): 0
1
Invalid move! Try again.

X | X |
---+---+---
  | O |
---+---+---
O |   | X

Player 0, enter row and column (0-2): 0
2

X | X | O
---+---+---
  | O |
---+---+---
O |   | X

Player 0 WINS!

-----
Process exited after 98.06 seconds with return value 0
Press any key to continue . . . |
```

7. Supplementary Activity

1. First I use a header `#include <iomanip>` so I can use the `setw()` the function of that is you can make them even or set the width, for my situation I put `setw(4)` to have 4 spaces so they will not look messy. Next I use a `const int` to make the value (10) easy to input and I named it `SIZE` so whenever I use that `int` I will put `SIZE` instead of `10`. Then I assign a another in which is the table (**two-dimensional array**), then I put `[SIZE][SIZE]` that means 10 rows and 10 columns, Next I initialized the **for loop** so the I will starts to 0 then it will repeat while `i` is less than 10, same as the `j` (outer loop) Then I Input the table so they can combine and compute, the function of that is the first `i` will go row by row then the second `j` will go column inside that row. Next I use `cout` to print a header then the two `\n` will make a two lines space before the table,

Then `cout` again because I use it as a space. Then for the third for loop the `j` will loop 1 to 10 then the `cout` will ensure that each number is printed with a fixed width of 4 spaces, then I put the `"|"` to make them more organized. Then use `cout` and put `\n` to create a blank line before and after, then I put `----` inside to divide them. Then the next for loop will handle the rows of the table, printing the row number on the left, and inside it, another for loop will go through the columns to display each value stored in `table[i][j]`. The use of `setw(4)` makes sure everything lines up well, and the `"|"` keeps them organized, then I used a `cout` again for `-----` to make them divide. Then return 0;

2. First, I used the header `#include <iostream>` so I can use `cin` and `cout` for input and output. Next, I set a `const int SIZE = 3` because the Tic-Tac-Toe board has 3 rows and 3 columns, and it's easier to use `SIZE` instead of typing 3 everywhere.

Then, I created a two-dimensional array `board[SIZE][SIZE]` to represent the board. Each cell in the board can hold a space ' ' if it's empty, or 'X' or 'O' if a player has placed a mark there.

After that, I made the function `displayBoard()` to show the board on the screen. I used a nested for loop: the outer loop goes through the rows (`i`) and the inner loop goes through the columns (`j`). I used `cout` to print each cell, and " |" to separate the columns. I also added "----+----+----" between rows to make it look like a real Tic-Tac-Toe grid.

Next, I made the function `isValidMove()` to check if a player's move is allowed. It checks that the row and column are inside the board (0 to 2) and that the chosen cell is empty ' '.

I also made the function `checkWin()` to see if a player has won. It checks all rows, all columns, and both diagonals. If any of them have the same player mark in all three cells, it returns `true`, which means that player wins.

In the `main()` function, I initialized the board to all spaces ' '. I set `currentPlayer` to 'X' to start the game, `moves = 0` to count the moves, and `gameOver = false` to track if the game has ended.

Then I created a while loop that runs while `moves` is less than 9 and `gameOver` is `false`. Inside the loop, I first called `displayBoard()` to show the current state of the board. Then I asked the current player to enter a row and column.

I used `isValidMove()` to check the move. If it is valid, I put the player's mark in the chosen cell and increased `moves` by 1. Then I called `checkWin()` to see if the current player won. If yes, I displayed the board again and printed that the player wins, then set `gameOver = true`. If all moves are done but no one won, I printed that it's a draw. If the game is not over, I switched the player from 'X' to 'O' or from 'O' to 'X'.

If the move was invalid, I printed "Invalid move! Try again." so the player can choose another cell.

Then `return 0`;

8. Conclusion

What I learned here is about the multiplication table, using `#include <iomanip>`, `setw()`, and the proper use of `const` integers, and for tic-tac-toe, about `void` and `bool`.

For the multiplication table, I understood it and was able to link it with previous activities and organize it with design. But for tic-tac-toe, I admit I struggled because I still didn't quite get how it works. I started this activity yesterday and only finished the multiplication table. I really took a long time with tic-tac-toe, but I have self-studied the topic, and I hope that in future activities I can do it with confidence.

9. Assessment Rubric